

VVD Success Events Schema

success_event_mapping (VVD)

Reference table linking VVD tactic IDs to event type codes and success levels.

tactic_id	event_type_cd	success_level	event_description
2024XXVCN	1	1	Card Acquisition
2024XXVDA	1	1	Card Acquisition
2024XXVDT	2	1	Card Activation
2024XXVUI	3	1	Card Usage
2024XXVUI	4	2	Wallet Provisioning
2024XXVUT	4	1	Wallet Provisioning
2024XXVUT	3	2	Card Usage
2024XXVAW	4	1	Wallet Provisioning
2024XXVAW	3	2	Card Usage

success_events (VVD)

Core event-level fact table. Each row represents one client success event detected by the vintage engine.

Schema

Column	Type	Description	Nullable
data_asset	string	Event source identifier	No
clnt_no	string	Client number	No
acct_no	string	Account / card number	Yes
lob	string	Line of business	No
prod	string	Product	No
sub_prod	string	Sub-product / event detail	Yes
open_amt	decimal	Transaction or open amount	Yes
event_dt	date	Date the success event occurred	No
event_attributes	JSON	Flexible metadata	Yes
event_type_cd	int	Event type code (FK to success_event_ma	No
event_category	string	Event category classification	No

Event Type Codes (VVD)

event_type_cd	data_asset	lob	prod	metric_code	source_table	date_field
1	card_acquisition	Digital Footprint	VVD	VVD_ACQ_001	DDWTA_VISA_DR_CR	ISS_DT
2	card_activation	Digital Footprint	VVD	VVD_ACT_001	DDWTA_VISA_DR_CR	ACTV_DT
3	card_usage	Payments	VVD	VVD_USG_001	DDWTA_T_PT_OF_SA	TXN_DT
4	wallet_provisioning	Digital Footprint	VVD	VVD_PRV_001	CLNT_CRD_POS_LOG	TXN_DT

Sample Rows

data_asset	clnt_no	acct_no	lob	prod	sub_prod	open_amt	event_dt	event_attribut	event_type_cd	event_category
card_acquisitio	123456789	555555	Digital Footprin	VVD	Virtual Visa De	null	2025-06-15	{"sts_cd": "06",	1	product_open
card_acquisitio	987654321	666666	Digital Footprin	VVD	Virtual Visa De	null	2025-07-01	{"sts_cd": "08",	1	product_open
card_activation	123456789	555555	Digital Footprin	VVD	Virtual Visa De	null	2025-06-20	{"sts_cd": "06",	2	product_activation
card_usage	123456789	555555	Payments	VVD	Standard Purcl	45.99	2025-06-25	{"txn_tp": 10, "r	3	product_usage

card_usage	123456789	555555	Payments	VVD	E-commerce P	129.00	2025-07-02	{"txn_tp": 13, "r	3	product_usage
card_usage	987654321	666666	Payments	VVD	Recurring / Sul	14.99	2025-07-10	{"txn_tp": 12, "r	3	product_usage
wallet_provisio	123456789	555555	Digital Footprin	VVD	Apple Pay	0	2025-06-22	{"token_wallet_	4	digital_wallet
wallet_provisio	987654321	666666	Digital Footprin	VVD	Google Pay	0	2025-07-05	{"token_wallet_	4	digital_wallet

Column Details

data_asset

Identifies the type of success event. Maps directly to the metric name in the vintage engine.

Value	Vintage Engine Key	Description
card_acquisition	SUCCESS_DEFINITIONS["card_acquisition"]	New VVD card issued
card_activation	SUCCESS_DEFINITIONS["card_activation"]	Existing VVD card activated
card_usage	SUCCESS_DEFINITIONS["card_usage"]	VVD card used for POS transaction
wallet_provisioning	SUCCESS_DEFINITIONS["wallet_provisioning"]	VVD card added to digital wallet

clnt_no

Client number. Extraction varies by source:

Source	Extraction
DDWTA_VISA_DR_CRD	Direct CLNT_NO column
DDWTA_T_PT_OF_SALE_TXN	SUBSTR(CLNT_CRD_NO, 7, 9) stripped of leading zero
CLNT_CRD_POS_LOG	CAST(SUBSTR(CLNT_CRD_NO, 7, 9) AS INTEGER)

sub_prod

For VVD events, sub_prod captures the specific event variant:

event_type_cd	sub_prod values
1 (acquisition)	Virtual Visa Debit
2 (activation)	Virtual Visa Debit
3 (usage)	Standard Purchase (TXN_TP=10), E-commerce Purchase
4 (provisioning)	Apple Pay, Google Pay, Samsung Pay (derived from t

open_amt

event_type_cd	open_amt meaning
1 (acquisition)	null (no amount at issuance)
2 (activation)	null (no amount at activation)
3 (usage)	AMT1 - transaction amount (must be > 0)
4 (provisioning)	0 (provisioning events are always zero-amount)

event_attributes (JSON)

Flexible metadata column. Contents vary by event type:

card_acquisition / card_activation (event_type_cd 1, 2):

{ "sts_cd": "06", "srv_id": 36 }

card_usage (event_type_cd 3):

{

```
"txn_tp": 10,  
"msg_tp": "0210",  
"srvc_cd": 36  
}
```

wallet_provisioning (event_type_cd 4):

```
{
  "token_wallet_ind": "Y",
  "bin": "45190",
  "pos_entry_mode": "000",
  "srvc_cd": 36
}
```

event_category

event_type_cd	event_category	Aligns with whiteboard
1	product_open	Product Open
2	product_activation	Product Enhancement
3	product_usage	Product Usage
4	digital_wallet	Product Enhancement

Relationship to campaign_mapping

How success_events connects to campaign success definitions:

campaign_mapping (Table C)			success_events (Table 1 green)	
mnemonic	1st_success		data_asset	event_type_cd
VCN	Card Acquisition	---->	card_acquisition	1
VDA	Card Acquisition	---->	card_acquisition	1
VDT	Card Activation	---->	card_activation	2
VUI	Card Usage	---->	card_usage	3
VUT	Wallet Provisioning	---->	wallet_provisioning	4
VAW	Wallet Provisioning	---->	wallet_provisioning	4

|

success_event_mapping (Table 1)
links tactic_id -> event_type_cd
with success_level (1st, 2nd)

Filter Logic Summary

All filters that must be applied when populating success events from source tables:

event_type_cd = 1 (Card Acquisition)

```
SOURCE: DDWTA_VISA_DR_CRD
WHERE STS_CD      IN ('06', '08')      -- Active or Approved
      AND SRVC_ID = 36                 -- VVD product
      AND ISS_DT   IS NOT NULL         -- Card was issued

OUTPUT:
      clnt_no    = CLNT_NO
      event_dt   = ISS_DT
      open_amt   = NULL
```

event_type_cd = 2 (Card Activation)

```
SOURCE: DDWTA_VISA_DR_CRD
WHERE STS_CD      IN ('06', '08')      -- Active or Approved
      AND SRVC_ID = 36                -- VVD product
      AND ISS_DT   IS NOT NULL         -- Card was issued first

OUTPUT:
  clnt_no  = CLNT_NO
  event_dt  = ACTV_DT
  open_amt = NULL
```

event_type_cd = 3 (Card Usage)

```
SOURCE: DDWTA_T_PT_OF_SALE_TXN
WHERE SRVC_CD = 36      -- VVD product
      AND AMT1 > 0      -- Real transaction
      AND (
        (TXN_TP = 10 AND MSG_TP = '0210') -- Standard purchase
        OR (TXN_TP = 13 AND MSG_TP = '0210') -- E-commerce
        OR (TXN_TP = 12 AND MSG_TP = '0220') -- Recurring
      )

OUTPUT:
  clnt_no  = SUBSTR(CLNT_CRD_NO, 7, 9) stripped of leading zeros
  event_dt  = TXN_DT
  open_amt  = AMT1
```

event_type_cd = 4 (Wallet Provisioning)

```
SOURCE: DDWV05.CLNT_CRD_POS_LOG B
JOIN DL_DECMAN.TOKEN_LIST C ON B.TOKN_REQSTR_ID = C.TOKEN_ID
WHERE B.AMT1 = 0      -- Provisioning (zero-amount)
      AND SUBSTR(B.CLNT_CRD_NO, 1, 5) = '45190' -- VVD card BIN
      AND SUBSTR(B.VISA_DR_CRD_NO,1,5) = '45199' -- VVD debit BIN
      AND SUBSTR(B.TOKN_REQSTR_ID,1,1) > '0'    -- Valid token requestor
      AND B.POS_ENTR_MODE_CD_NON_EMV = '000'    -- Token provisioning mode
      AND B.SRVC_CD = 36      -- VVD product
      AND C.TOKEN_WALLET_IND = 'Y'      -- Wallet confirmed

OUTPUT:
  clnt_no  = CAST(SUBSTR(B.CLNT_CRD_NO, 7, 9) AS INTEGER)
  event_dt  = B.TXN_DT
  open_amt  = 0
```

Cross-Sell View (VVD Contribution)

When VVD success_events are pivoted into the cross-sell crosstab (Table 5), VVD contributes to the cards column (or a new digital column if VVD is tracked separately):

data_asset	clnt_no	hef	investments	cards	pba	loans	digital_footprint	event_dt
cross-sell	123456789	0	0	0	0	0	1	2025-06-15
cross-sell	987654321	0	0	0	0	0	1	2025-07-01

Open question: Should VVD events roll up under the existing cards column or be tracked in a separate digital_footprint column in the cross-sell view?